

GSA Gateway

Technical Reference for New Contributors

New Jersey Institute of Technology
Graduate Student Association

Discord Bot + Static Website

Author: Mohammad Dindoost

Role: VP Academic Affairs, Ph.D. Candidate in Computer Science

Contact: md724@njit.edu

Repository: gsa-gateway

Branch: main

Document Version: June 2026

This document is intended for new contributors, student developers, and researchers who want to understand how GSA Gateway is built, why specific technology choices were made, and how to extend the system.

Contents

1	Project Overview	3
1.1	What Problems Does It Solve?	3
1.2	Technology Stack at a Glance	3
2	Repository Structure	3
2.1	The bot/commands/ Directory	4
2.2	The bot/services/ Directory	5
2.3	The bot/data/ Directory	5
3	System Architecture	6
3.1	Startup Sequence	6
3.2	Message Routing Flow	7
4	The RAG Pipeline — End to End	8
4.1	Offline Phase: Building the Index	8
4.2	Online Phase: Answering a Question	8
4.3	The Embedding Asymmetry	10
4.4	Chunking Strategy	10
4.4.1	Markdown FAQ Files (chunk_markdown_faq)	10
4.4.2	Markdown Policy Files (chunk_markdown_policy)	10
4.4.3	YAML Events (chunk_yaml_events)	10
4.4.4	YAML Contacts (chunk_yaml_contacts)	11
4.4.5	YAML Resources (chunk_yaml_resources)	11
4.5	Conversation Memory	11
4.6	Query Expansion	11
5	Vector Database: ChromaDB	11
5.1	What is a Vector Database?	12
5.2	Why ChromaDB?	12
5.3	ChromaDB in Our Codebase	12
5.4	Alternatives to ChromaDB	13
5.5	The Distance Metric: Cosine Similarity	14
6	The Embedding Model: nomic-embed-text	15
6.1	Why This Model?	15
6.2	Alternatives	15
7	The LLM: Ollama + llama3.1:8b	15
7.1	What Ollama Is	15
7.2	The System Prompt Design	15
7.3	Graceful Degradation	16
8	The Relational Database: SQLite	16

8.1	Tables	16
8.2	Privacy Design	16
9	Fuzzy Search: rapidfuzz	16
10	Configuration System	17
11	Scripts	18
12	The Static Website	18
13	MathCafe Feature	19
14	Security and Privacy Design	19
15	Running the System: Quick Reference	19
15.1	First-Time Setup	19
15.2	After Editing Knowledge Base Files	20
15.3	Health Check	20
16	Testing	20
17	Key Invariants for Contributors	21
18	Architecture Decision Log	21
19	Glossary	22

1 Project Overview

GSA Gateway is the official digital infrastructure of the NJIT Graduate Student Association. It consists of two independently deployable components:

1. **Discord Bot** — A Python bot that answers student questions using a full Retrieval-Augmented Generation (RAG) pipeline, manages slash commands for events, resources, feedback, and travel funding, and runs automated daily digests.
2. **Static Website** — A pure HTML/CSS/JS website hosted on GitHub Pages that surfaces the same information (events, officers, resources) to students without Discord.

1.1 What Problems Does It Solve?

Before GSA Gateway, students had no single place to answer questions like:

- “How do I apply for a travel award and what is the deadline?”
- “Who is the GSA VP of Finances and how do I reach them?”
- “What events are happening this month?”
- “What is the reimbursement cap for international travel?”

Officers answered these repetitively via email and Discord messages. GSA Gateway automates this by grounding every answer in official documents and routing questions intelligently.

1.2 Technology Stack at a Glance

Component	Technology	Why
Bot framework	discord.py 2.x	Mature, async-native, slash command support
Language	Python 3.11+	Async ecosystem, ML libraries
LLM (generation)	Ollama + llama3.1:8b	Local, no API cost, privacy-safe
Embedding model	nomic-embed-text	Open source, strong semantic quality
Vector database	ChromaDB	Persistent, zero-server, Python-native
Relational database	SQLite	Simple, file-based, no infrastructure
Fuzzy search	rapidfuzz	Fast approximate string matching
Tokenizer	tiktoken (cl100k_base)	Accurate token counting for chunking
Website	HTML/CSS/JS	No framework, GitHub Pages compatible

2 Repository Structure

```

1 gsa-gateway/
2 + - bot/                                # All Python bot code
3 |   + - main.py                         # Entry point - GSABot class, startup sequence
4 |   + - config.py                       # Reads .env → typed Config dataclass
5 |   + - commands/                       # Discord slash commands (cogs)

```

```

6 | + - services/           # Business logic, RAG pipeline, database
7 | \ - data/              # Knowledge base files (Markdown + YAML)
8 + - website/            # Static site
9 | + - *.html             # One file per page
10 | + - assets/            # style.css, app.js
11 | \ - data/events.json   # Auto-generated from events.yml
12 + - scripts/            # Utility scripts (setup, index build, export)
13 + - docs/               # Architecture and deployment documentation
14 + - chroma_db/          # ChromaDB persistent store (auto-created)
15 + - gsa_gateway.db       # SQLite database (auto-created)
16 + - gsa_gateway.log      # Runtime log (auto-created)
17 \ - .env                # Local secrets (never committed)

```

Listing 1: Top-level directory layout

2.1 The `bot/commands/` Directory

Each file here is a Discord *cog* — a self-contained class that registers one or more slash commands. Adding a new command means creating a new file and listing it in `EXTENSIONS` in `main.py`. No other wiring needed.

File	Responsibility
<code>ask.py</code>	<code>/ask</code> — fuzzy-searches knowledge base, optional Ollama LLM answer
<code>chat.py</code>	<code>on_message</code> handler — free-form chat in <code>#ask-gsa</code> and DMs; full RAG pipeline
<code>events.py</code>	<code>/events</code> (list) and <code>/event [name]</code> (detail)
<code>initiative.py</code>	<code>/initiative</code> — Discord Modal collecting 5 fields, stored to SQLite
<code>feedback.py</code>	<code>/feedback</code> — anonymous message stored to SQLite
<code>resources.py</code>	<code>/resources [category]</code> — lists YAML-sourced resources
<code>contact.py</code>	<code>/contact [role]</code> — directory lookup from <code>contacts.yml</code>
<code>help_cmd.py</code>	<code>/help</code> — ephemeral embed with full command reference
<code>admin.py</code>	<code>/admin_summary</code> , <code>/admin_export</code> , <code>/admin_stats</code> , <code>/admin_announce</code> , <code>/admin_add_event</code> , <code>/admin_rebuild_index</code> (all ephemeral, admin role gated)
<code>qrcode_cmd.py</code>	<code>/qrcode</code> — generates GSA-branded QR code with color options

2.2 The `bot/services/` Directory

Services are the business logic layer. They have no knowledge of Discord; they receive plain Python objects and return plain Python objects. This separation makes them independently testable.

File	Responsibility
<code>chunker.py</code>	Splits all KB files into ≤ 350 -token chunks for vector storage
<code>embedder.py</code>	Converts text to vectors using Ollama's <code>nomic-embed-text</code>
<code>vector_store.py</code>	Wraps ChromaDB — stores chunks, queries by cosine similarity
<code>bm25_index.py</code>	BM25Okapi lexical index over all chunks — exact/sparse term retrieval
<code>retriever.py</code>	Hybrid: vector + BM25 in parallel, RRF fusion, rerank to top-7, expand siblings
<code>ollama_client.py</code>	Sends prompts to Ollama LLM, builds system/user prompt from chunks
<code>conversation.py</code>	Per-user in-memory session history (60 min timeout, 5-turn window)
<code>intent_detector.py</code>	Classifies messages — greeting, food, thanks, help, question, statement
<code>knowledge_base.py</code>	Loads all <code>.md</code> and <code>.yaml</code> data files into memory at startup
<code>search.py</code>	<code>rapidfuzz</code> fuzzy search over KB entries (threshold: 60.0)
<code>database.py</code>	All SQLite CRUD operations, privacy hashing, statistics
<code>moderation.py</code>	<code>RateLimiter</code> (5 calls/60s), channel allowlist, admin role check
<code>summaries.py</code>	<code>weekly_summary()</code> — generates markdown summary from SQLite data
<code>mathcafe.py</code>	Posts a daily math fact to <code>#gsa-mathcafe</code> ; tracks Maryam Mirzakhani birthday
<code>scheduler.py</code>	Daily digest cog — posts announcements on schedule

2.3 The `bot/data/` Directory

This is the knowledge base. Every fact the bot can answer comes from one of these files. Editing a Markdown or YAML file here changes what the bot knows. After editing, you must rebuild the vector index.

File	Format	Content
gsa_faq.md	Markdown FAQ	12+ Q&A pairs about GSA services
mmi_workshop.md	Markdown FAQ	Full MMI 2025 and 2026 knowledge base
bot_features.md	Markdown FAQ	Bot command reference for #ask-gsa
gsa_constitution.md	Markdown policy	GSA constitution and bylaws
travel_award.md	Markdown policy	Travel award process, tiers, Chrome River
club_finance.md	Markdown policy	Club financial bylaws
rules.md	Markdown policy	Community guidelines
events.yml	YAML	5+ events with dates, locations, descriptions
contacts.yml	YAML	9 officers and campus offices with emails
resources.yml	YAML	8 categories × 3–4 resources each

Markdown FAQ Format

All .md FAQ files must follow this exact format so the chunker can parse them correctly:

```
1 ## Q: Your question here?
2 **A:** Your answer here (can span multiple lines).
```

3 System Architecture

3.1 Startup Sequence

When the bot starts, `GSABot.setup_hook()` runs before Discord marks the bot as online. This ensures all services are ready before any user message arrives.

```
1 async def setup_hook(self):
2     # 1. Relational database
3     self.db = Database(config.database_path)
4     self.db.connect()
5     self.db.init_tables()
6
7     # 2. Knowledge base (loads .md and .yaml files into memory)
8     self.kb = KnowledgeBase(data_dir=config.data_dir)
9     self.kb.load()
10
11     # 3. Fuzzy search over loaded KB
12     self.search_svc = SearchService(self.kb)
13
14     # 4. Rate limiter (in-memory, resets on restart)
15     self.rate_limiter = RateLimiter(max_calls=5, period_seconds=60)
16
17     # 5. LLM client (only if OLLAMA_ENABLED=true)
18     self.ollama = OllamaClient(...)
```

```

19
20     # 6. Conversation manager (in-memory per-user sessions)
21     self.conversation_manager = ConversationManager(...)
22
23     # 7. Embedding service (nomic-embed-text via Ollama)
24     self.embedder = EmbeddingService(...)
25
26     # 8. ChromaDB vector store (reads from ./chroma_db/)
27     self.vector_store = VectorStore(db_path=config.chroma_db_path)
28
29     # 9. Retriever (ties embedder + vector store together)
30     self.retriever = Retriever(embedder=self.embedder,
31                                vector_store=self.vector_store)
32
33     # 10. Intent detector
34     self.intent_detector = IntentDetector()
35
36     # 11. Load all cogs (commands)
37     for ext in EXTENSIONS:
38         await self.load_extension(ext)

```

Listing 2: Startup sequence (simplified from main.py)

All services are attached to the bot instance. Every cog accesses them via `self.bot.db`, `self.bot.retriever`, etc.

3.2 Message Routing Flow

When a student sends a message in `#ask-gsa` (or DMs the bot), the `ChatCog.on_message` handler processes it through a series of gates:

```

1 Gate 1: Is this the bot's own message?      → ignore
2 Gate 2: Is this a slash command?           → ignore (handled by cog)
3 Gate 3: Does message mention another user?  → ignore (avoid cross-talk)
4 Gate 4: Is it the ask-gsa channel or DM?    → proceed | ignore
5 Gate 5: IntentDetector classifies intent
6   - greeting      → warm reply, no RAG
7   - thanks        → acknowledgement, no RAG
8   - food          → retrieve events with food tag
9   - social        → retrieve social events
10  - clear         → wipe session history
11  - help         → show command list
12  - question/statement → full RAG pipeline
13 Gate 6: Rate limiter check (5 req/60s)     → throttle if exceeded
14 Gate 7: RAG pipeline                       → generate answer

```

Listing 3: Message routing gates (chat.py)

4 The RAG Pipeline — End to End

Retrieval-Augmented Generation (RAG) is the core intelligence of the bot. Instead of fine-tuning a model on GSA data (expensive, hard to update), RAG retrieves relevant documents at query time and injects them into the LLM prompt. The LLM is explicitly instructed to answer *only* from the provided documents, preventing hallucination.

4.1 Offline Phase: Building the Index

This runs once (or after any KB file edit) via `python scripts/build_index.py -reset`.

```
Knowledge Base Files
(.md, .yaml in bot/data/)
|
v
[DocumentChunker]
Splits each file into ≤ 350-token chunks
Each chunk: text + source_file + section_title + metadata
|
v
[EmbeddingService]
Calls Ollama /api/embed with "search_document: {text}"
Model: nomic-embed-text | Output: 768-dimensional float vector
|
v
[VectorStore (ChromaDB)]
Stores: chunk_id, text, embedding, metadata
Location: ./chroma_db/ (persistent, survives restart)
143 chunks total from 10 source files
```

Figure 1: Offline indexing pipeline

4.2 Online Phase: Answering a Question

This runs for every student message that reaches the RAG path.

```

Student message: "How many speakers at MMI 2026 from Rutgers?"
|
v
[OllamaClient.expand_query()]                (optional)
Short/vague queries are rewritten:
"MMI" → "What is the MMI Workshop Series at NJIT?"
|
v
[Retriever._build_search_query()]
Cleans mentions/channels, optionally appends
topic keywords from conversation history
|
v
PARALLEL:
[EmbeddingService.embed_query()]              [BM25Index.search()]
Semantic vector search                        Lexical exact-term search
"search_query: " prefix                      tokenize + BM25Okapi scores
top-20 by cosine similarity                  top-20 by BM25 score
|                                           |
+ -- -- -- -- -- +   + -- -- -- -- -- +
|                       |
v                       v
[Retriever._reciprocal_rank_fusion()]
Merge both ranked lists: score = sum(1 / (60 + rank))
Normalize fused scores to [0.40, 0.95] → stored as rrf_score
BM25-only chunks (unknown terms, sparse matches) are included
Result: up to 40 unique candidates
|
v
[Retriever._rerank()]
base_score   = rrf_score (not raw cosine - fair for BM25 chunks)
common_bonus = word-boundary hits * 0.05 (max common contribution)
proper_bonus = capitalized non-stopword hits * 0.15 (3x weight:
               names like "Singh", "MARCuS" are key disambiguators)
source_bonus = faq:0.05, policy:0.03, event:0.02
title_bonus  = 0.10 if query word in section title
Keeps top-7 by relevance_score
|
v
[Retriever._include_siblings()]
For any chunk tagged with qa_base_id:
fetch all continuation parts of split Q&A answers
|
v
[OllamaClient.generate_answer()]
SYSTEM: GSA Gateway persona + 11 rules (incl. exhaustive enumeration)
       + conversation history
USER:   [Document 1...Document N] + student question
Calls Ollama /api/generate (llama3.1:8b, temp=0.3)
|
v
Discord reply with source attribution
"According to Document 2: MMI Workshop Series..."

```

9
Figure 2: Online hybrid retrieval and generation pipeline (as of June 2026)

4.3 The Embedding Asymmetry

A subtle but important design decision: documents and queries are embedded with different prefixes. This is specific to `nomic-embed-text`:

Case	Prefix added before embedding
Indexing a document chunk	"search_document: {text}"
Embedding a user query	"search_query: {text}"

This is called *asymmetric dense retrieval*. The model was trained to align query vectors with document vectors even when the phrasing differs. Without the correct prefix, retrieval quality degrades noticeably.

4.4 Chunking Strategy

The `DocumentChunker` uses different strategies per file type:

4.4.1 Markdown FAQ Files (`chunk_markdown_faq`)

Splits on `##` headings. Each Q&A pair becomes one chunk. If the full "Question: ... \nAnswer: ..." text exceeds 350 tokens, the answer is split using `split_text_by_tokens()`, which:

- Splits text into sentences (on `. ! ?` boundaries and newlines)
- Accumulates sentences until the 350-token limit is approached
- Carries a 50-token overlap into the next chunk for context continuity
- Tags multi-part chunks with `qa_base_id` and `chunk_index` metadata so the retriever can always include all parts of a split answer

4.4.2 Markdown Policy Files (`chunk_markdown_policy`)

Splits on `##` headings. Each section becomes one chunk. If a section exceeds 350 tokens, it is sub-chunked with the same sentence-aware splitter. Section titles are prepended to each sub-chunk for context.

4.4.3 YAML Events (`chunk_yaml_events`)

One chunk per event, formatted as structured text:

```
Event: MMI Workshop 2026
Date: 2026-05-21
Location: NJIT GITC 1100
Description: ...
```

4.4.4 YAML Contacts (`chunk_yaml_contacts`)

One chunk per officer entry, one chunk per campus office entry. Each contains role, name, email, responsibilities.

4.4.5 YAML Resources (`chunk_yaml_resources`)

One chunk per resource item across all categories. Each chunk includes the category name for context.

4.5 Conversation Memory

`ConversationManager` maintains per-user session history entirely **in memory** — there is no database table for conversations. This is intentional: session data is private by design and does not need to persist across restarts.

Setting	Value (configurable in <code>.env</code>)
Session timeout	60 minutes of inactivity
Max turns kept	5 user+assistant pairs (10 total turns)
Cleanup interval	Every 5 minutes (background async task)
Storage	Python dict in-memory, never on disk

When conversation history exists, two things change:

1. The retriever appends up to 5 topic keywords from recent assistant responses to the search query, improving follow-up retrieval.
2. The LLM system prompt is extended with the last 5 turns so it can resolve references like “what about step 2?” or “how much is that?”

4.6 Query Expansion

When a student types something very short or ambiguous (e.g., “MMI”, “funding”, “travel”), a preliminary call to `expand_query()` rewrites it to a full question before retrieval. This prevents poor embedding alignment from a single token query.

```
Input:  "MMI"
Output: "What is the MMI Workshop Series at NJIT?"

Input:  "travel"
Output: "How can I apply for a GSA travel award?"
```

The expansion uses `llama3.1:8b` with temperature 0.1 and `num_predict=60`, making it fast. If Ollama is unavailable, the original query is used as-is.

5 Vector Database: ChromaDB

5.1 What is a Vector Database?

A vector database stores high-dimensional float vectors (embeddings) and retrieves the nearest neighbours by distance metric. In this system:

- Every KB chunk is converted to a 768-dimensional vector by `nomic-embed-text`
- At query time, the student's question is also converted to a vector
- The database returns the chunks whose vectors are closest to the query vector (cosine similarity)
- Semantic similarity is captured, not just keyword matching

5.2 Why ChromaDB?

Reason	Detail
No server required	ChromaDB runs in-process. You do not need to start a separate database server. For a bot running on a single machine, this eliminates an entire infrastructure dependency.
Persistent by default	<code>chromadb.PersistentClient(path="./chroma_db")</code> writes to disk automatically. The index survives restarts without any extra configuration.
Python-native API	Direct Python calls with no query language. Adding chunks, querying, filtering, and resetting are all single function calls.
Metadata filtering	Supports <code>where={"source_type": "faq"}</code> style filters, which we use to restrict food queries to event chunks only.
Small-to-medium scale	For 143 chunks (our current size), ChromaDB uses HNSW indexing with cosine space, which is extremely fast. Even at 10,000 chunks it remains suitable.
Open source, free	MIT license. No usage limits, no API keys.

5.3 ChromaDB in Our Codebase

The `VectorStore` class (`bot/services/vector_store.py`) wraps all ChromaDB interaction:

```

1 class VectorStore:
2     def add_chunks(chunks, embeddings)

```

```

3      # Bulk add during index build
4
5      def query(query_embedding, n_results=20, source_type_filter=None)
6          # Vector similarity search, returns list[dict] with similarity scores
7
8      def get_all_chunks()
9          # Return every chunk - used to build the BM25 index at startup
10
11     def get_sibling_chunks(qa_base_id)
12         # Fetch all continuation parts of a split Q&A answer
13
14     def get_chunk_count()
15         # Total chunks in the collection
16
17     def reset()
18         # Delete and recreate collection (used by build_index.py - reset)

```

Listing 4: Key VectorStore methods

ChromaDB stores three things per chunk:

1. **ID** — unique string identifier (e.g., mmi_workshop_faq_7_0_1)
2. **Document** — the raw chunk text
3. **Metadata** — structured fields: `source_file`, `source_type`, `section_title`, `token_count`, and optionally `qa_base_id` + `chunk_index` for multi-part answers

5.4 Alternatives to ChromaDB

Understanding why we chose ChromaDB requires understanding the alternatives.

Database	Strengths	Weaknesses	Best for
ChromaDB	No server, persistent, Python-native, free	Not suitable for multi-node or billion-scale	Our use case
FAISS (Meta)	Extremely fast, industry-proven, supports GPU indexing	In-memory only by default (no built-in persistence), no meta-data filtering, lower-level API	High-performance search where you manage persistence yourself
Qdrant	High performance, rich filtering, Rust internals, can run as embedded or server	Slightly more complex setup than ChromaDB	Production systems needing advanced filters

Weaviate	GraphQL API, multi-modal, schema-driven	Requires running a separate server, heavier infrastructure	Enterprise, multi-modal (image+text) retrieval
Pinecone	Fully managed cloud, serverless option, scales automatically	Paid after free tier, data leaves your infrastructure	Teams that do not want to manage infrastructure
Milvus	Billion-scale, distributed, battle-tested at Zilliz	Heavy infrastructure (requires etcd, MinIO, Pulsar)	Large-scale enterprise with dedicated DevOps
pgvector	Lives inside PostgreSQL, SQL-native, easy to join with relational data	Requires PostgreSQL, slower than dedicated vector DBs at scale	Teams already running PostgreSQL

When you would migrate away from ChromaDB

If the knowledge base grows beyond ~50,000 chunks, or if you need to deploy on multiple machines, Qdrant (embedded mode) or pgvector are the natural next steps. Both can be swapped in by replacing only `vector_store.py` — no other code changes needed.

5.5 The Distance Metric: Cosine Similarity

ChromaDB is configured with `"hnsw:space": "cosine"`. Cosine similarity measures the angle between two vectors, ignoring magnitude:

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

A score of 1.0 means the vectors point in exactly the same direction (identical semantic meaning). A score of 0.0 means perpendicular (unrelated). ChromaDB returns distances (`1.0 - similarity`), which we convert back: `similarity = 1.0 - distance`.

Our minimum similarity threshold is **0.3**. Chunks below this are discarded before reranking.

6 The Embedding Model: nomic-embed-text

`nomic-embed-text` is an open-source text embedding model developed by Nomic AI. It produces 768-dimensional vectors and is specifically designed for retrieval tasks.

6.1 Why This Model?

- Runs locally via Ollama — no API key, no network call, no cost
- Strong performance on standard retrieval benchmarks (BEIR suite)
- Supports asymmetric retrieval via task prefixes (`search_query:` vs `search_document:`)
- Fast enough to embed 143 chunks in under 1 second on a CPU

6.2 Alternatives

Model	Dimensions	Local?	Cost
<code>nomic-embed-text</code> (ours)	768	Yes (Ollama)	Free
<code>text-embedding-3-small</code> (OpenAI)	1536	No	\$0.02/1M tokens
<code>text-embedding-3-large</code> (OpenAI)	3072	No	\$0.13/1M tokens
<code>mxbai-embed-large</code>	1024	Yes (Ollama)	Free
<code>all-MiniLM-L6-v2</code> (sentence-transformers)	384	Yes	Free

To swap the embedding model, change `EMBEDDING_MODEL` in `.env` and rebuild the index. The vector store must be fully rebuilt because vector dimensions and semantic spaces differ between models.

7 The LLM: Ollama + llama3.1:8b

7.1 What Ollama Is

Ollama is a local model server that downloads and serves open-weight language models. The bot talks to it via HTTP on `localhost:11434`.

```
1 POST /api/generate # Text generation (question answering, query expansion)
2 POST /api/embed    # Text embedding (used by EmbeddingService)
```

Listing 5: Ollama endpoints used

7.2 The System Prompt Design

`OllamaClient` constructs a two-part prompt for every RAG answer:

System prompt (always present):

- Defines the GSA Gateway persona
- Lists 10 explicit rules (never hallucinate, cite documents, format for Discord, etc.)

- Appends conversation history when a session exists

User prompt (built per query):

- Injects retrieved chunks as numbered documents with source names and relevance scores
- Appends the student’s question
- Instructs the model to answer only from the provided documents

Generation parameters: `temperature=0.3` (low randomness for factual answers), `top_p=0.9`, `num_predict=512` (max response tokens).

7.3 Graceful Degradation

If Ollama is unavailable or times out (60-second default), the bot falls back to the `SearchService` fuzzy-search answer. If fuzzy search also yields no result above 60% confidence, the bot returns a polite referral message to a GSA officer. The system never silently fails.

8 The Relational Database: SQLite

SQLite stores everything that needs to persist across restarts and that has structured, queryable form.

8.1 Tables

Table	Contents
<code>feedback</code>	Anonymous student feedback submissions
<code>initiatives</code>	Initiative proposals (5-field Modal form)
<code>events</code>	Events added via <code>/admin_add_event</code>
<code>message_stats</code>	Per-channel message counts for weekly summaries

8.2 Privacy Design

User IDs are never stored in plain text. Before any database write, `hash_user_id(user_id)` applies a SHA-256 hash (with a secret salt from `.env`). This means GSA officers can see aggregate statistics (“12 feedback submissions this week”) but cannot identify which student submitted which entry.

9 Fuzzy Search: rapidfuzz

`SearchService` (`bot/services/search.py`) provides keyword-based approximate matching as a fallback when RAG is unavailable or as a complement to it in the `/ask` command.

rapidfuzz uses the WRatio algorithm (Weighted Ratio), which combines multiple string similarity measures and returns a score from 0–100. The confidence threshold is **60.0**: matches below this are discarded and a fallback message is shown.

```
1 # In search.py
2 result = process.extractOne(query, candidates, scorer=fuzz.WRatio)
3 if result and result[1] ≥ 60.0:
4     return best_match
5 else:
6     return None # triggers fallback message
```

Listing 6: How the threshold works

10 Configuration System

All configuration lives in a `.env` file (never committed to git). The Config dataclass in `bot/config.py` reads these at startup and provides typed access throughout the codebase.

```
1 # Required
2 DISCORD_TOKEN=your_token_here
3 DISCORD_GUILD_ID=your_guild_id
4
5 # Ollama
6 OLLAMA_ENABLED=true
7 OLLAMA_MODEL=llama3.1:8b
8 OLLAMA_URL=http://localhost:11434
9 OLLAMA_TIMEOUT=60
10 EMBEDDING_MODEL=nomic-embed-text
11
12 # Bot behavior
13 ADMIN_ROLE_NAME=GSA Officer
14 ALLOWED_CHANNELS=ask-gsa,gsa-bot-testing
15 ASK_GSA_CHANNEL=ask-gsa
16 BOT_PREFIX=gsa
17
18 # RAG / vector store
19 CHROMA_DB_PATH=./chroma_db
20 CONVERSATION_TIMEOUT_MINUTES=60
21 CONVERSATION_MAX_TURNS=5
22
23 # Database and privacy
24 DATABASE_PATH=./gsa_gateway.db
25 USER_ID_SALT=your_random_salt_here
26
27 # Channel names (must match Discord channel names exactly)
28 CHANNEL_ANNOUNCEMENTS=gsa-announcements
29 CHANNEL_EVENTS=gsa-events
30 CHANNEL_FOOD=gsa-food
31 MATHCAFE_CHANNEL=gsa-mathcafe
```

Listing 7: Key `.env` variables

11 Scripts

Script	Purpose
<code>scripts/setup.sh</code>	Full first-time setup: creates venv, installs dependencies, initialises SQLite tables, runs tests. Run once on a new machine.
<code>scripts/build_index.py</code>	Rebuilds the ChromaDB vector index from all <code>bot/data/</code> files. Run after any KB file edit. <code>-reset</code> flag forces full rebuild without prompting.
<code>scripts/run_bot.sh</code>	Starts the bot process with log output to <code>gsa_gateway.log</code> .
<code>scripts/restart.sh</code>	Stops any running bot process, optionally restarts Ollama, and starts the bot fresh. Safe to run during active sessions.
<code>scripts/health_check.sh</code>	Checks whether the bot process is running, how long it has been up, memory usage, and MathCafe last-post status.
<code>scripts/export_events_json.py</code>	Syncs <code>events.yml</code> to <code>website/data/events.json</code> for the static website. Run after adding events.
<code>scripts/export_weekly_summary.py</code>	Generates and prints the 7-day admin summary to stdout or a file.
<code>scripts/init_db.py</code>	Creates SQLite tables. Safe to re-run (idempotent).

12 The Static Website

The website at `website/` is pure HTML, CSS, and JavaScript with no build step and no framework. It is designed to be hosted on GitHub Pages.

File	Content
<code>index.html</code>	Hero section, feature cards, slash command table
<code>about.html</code>	GSA mission, officer bios
<code>events.html</code>	Loads and renders <code>data/events.json</code> via <code>fetch()</code>
<code>initiatives.html</code>	Call to action for using <code>/initiative</code> in Discord
<code>resources.html</code>	Static resource listings by category
<code>contact.html</code>	Officer and campus office directory
<code>assets/style.css</code>	NJIT red #CC0000 + dark gray, mobile-first, no frameworks
<code>assets/app.js</code>	Nav toggle, events loader, HTML escaping for XSS safety
<code>data/events.json</code>	Auto-exported from <code>events.yml</code> by the export script

Keeping events in sync

The website reads `events.json`, not `events.yml`. After editing `events.yml`, you must run `python scripts/export_events_json.py` and commit the updated `events.json`. If you forget, the website will show stale events.

13 MathCafe Feature

`MathCafeService` (`bot/services/mathcafe.py`) posts a daily mathematics fact to the `#gsa-mathcafe` Discord channel. Facts are drawn from a queue in `bot/data/mathcafe/facts.y`

Special behavior: on **May 12th** (Maryam Mirzakhani's birthday), the bot posts a dedicated tribute fact using `bot/data/mathcafe/images/Maryam.png` and a biographical message regardless of the regular queue. It tracks `last_posted_year` to ensure this fires exactly once per year.

14 Security and Privacy Design

1. **User IDs are hashed.** No Discord user ID is ever stored in plain text. SHA-256 with a secret salt is applied in `database.py:hash_user_id()`.
2. **Admin commands are ephemeral.** All responses in `admin.py` use `ephemeral=True`, meaning only the requesting officer sees them.
3. **Rate limiting** prevents abuse: 5 RAG requests per user per 60 seconds.
4. **Channel allowlist** restricts bot responses to named channels only.
5. **LLM grounding.** The system prompt explicitly forbids the LLM from answering outside the provided context, preventing prompt injection from fabricating GSA policies.
6. **Admin role gate.** Admin commands check the `ADMIN_ROLE_NAME` Discord role before executing.
7. **Message gate 3.** Messages that mention another user (via `message.mentions` or raw `<@...>`) are ignored when the bot was not directly mentioned, preventing the bot from joining cross-user conversations uninvited.

15 Running the System: Quick Reference

15.1 First-Time Setup

```

1 # 1. Clone and enter the repo
2 git clone <repo-url> gsa-gateway
3 cd gsa-gateway
4
5 # 2. Run the setup script (creates venv, installs deps, inits DB)
```

```

6 bash scripts/setup.sh
7
8 # 3. Copy and fill in the environment file
9 cp .env.example .env
10 # Edit .env with your DISCORD_TOKEN and settings
11
12 # 4. Pull required Ollama models
13 ollama pull llama3.1:8b
14 ollama pull nomic-embed-text
15
16 # 5. Build the vector index
17 python3 scripts/build_index.py -reset
18
19 # 6. Start the bot
20 bash scripts/run_bot.sh

```

15.2 After Editing Knowledge Base Files

```

1 # Rebuild vector index (required after any .md or .yaml edit in bot/data/)
2 python3 scripts/build_index.py -reset
3
4 # If you added events, also sync the website
5 python3 scripts/export_events_json.py
6
7 # Restart the bot to reload the knowledge base into memory
8 bash scripts/restart.sh

```

15.3 Health Check

```

1 bash scripts/health_check.sh           # Process status, uptime, memory
2 tail -f gsa_gateway.log                # Live log stream
3 # In Discord (admin only):
4 /admin_stats                           # RAG chunk count, active sessions

```

16 Testing

Tests live in `bot/tests/` and use `pytest`. They operate on in-memory fixtures and do not require Ollama or Discord to be running.

File	What It Tests
<code>conftest.py</code>	Fixtures: in-memory SQLite, KB, SearchService
<code>test_search.py</code>	Fuzzy search hits, misses, confidence thresholds
<code>test_database.py</code>	CRUD operations, privacy hashing, aggregate stats
<code>test_commands.py</code>	Admin role check, rate limiter, channel allowlist

```

1 # Run all tests
2 pytest bot/tests/ -v
3

```

```
4 # Run a specific test file
5 pytest bot/tests/test_search.py -v
```

17 Key Invariants for Contributors

These are rules the system depends on. Breaking them causes silent failures, not errors.

Must-know rules

1. **Never store raw user IDs.** Always call `hash_user_id()` before writing to the database.
2. **Admin commands must use `ephemeral=True`.** Officer-only data should never be visible to the whole channel.
3. **Rebuild the index after any KB edit.** The ChromaDB store does not watch files; it must be explicitly rebuilt.
4. **The embedding prefix matters.** Documents use `"search_document: "`, queries use `"search_query: "`. Swapping them degrades retrieval silently.
5. **Conversation sessions are in-memory only.** They reset on restart. This is a feature, not a bug.
6. **The LLM never answers without context.** If `chunks` is empty, `generate_answer()` returns `None` and the bot falls back to fuzzy search.
7. **All config comes from `.env`.** Never hardcode tokens, paths, or channel IDs in Python files.

18 Architecture Decision Log

This section explains *why* specific choices were made, so future contributors do not reverse them accidentally.

1. **Why SQLite and not PostgreSQL?** The bot runs on a single machine with one writer at a time (the bot process). SQLite is sufficient, requires no server, and makes deployment trivial. If the bot is ever deployed across multiple servers, PostgreSQL would be the migration target.
2. **Why Ollama and not OpenAI API?** Two reasons: cost (the bot serves students continuously; API calls accumulate) and privacy (student questions about visa status, mental health resources, and finances should not leave the server). Local inference with `llama3.1:8b` covers the quality bar for factual Q&A.
3. **Why ChromaDB and not FAISS?** FAISS does not persist to disk natively and has no metadata filtering. ChromaDB does both out of the box. For 143 chunks, performance difference is immeasurable.
4. **Why `nomic-embed-text` and not a sentence-transformer?** It runs via the same Ollama endpoint already in use for generation, keeping infrastructure uniform. It also

natively supports asymmetric retrieval prefixes, which measurably improves recall for short student queries against longer document chunks.

5. **Why 350-token chunk limit?** Empirically chosen to balance two competing forces: embedding quality degrades for very long texts (over ~ 500 tokens), and retrieval misses context if chunks are too short (under ~ 100 tokens). 350 tokens keeps the typical Q&A answer together while fitting within the model’s sweet spot.
6. **Why sibling chunk expansion (added June 2026)?** Long Q&A answers that exceed 350 tokens are split into continuation chunks. The reranker would sometimes drop the second part because the first part scored higher (more keyword hits). The sibling system ensures that whenever any part of a split answer is retrieved, all parts are included. This fixed a bug where Prof. Enrique Dunn (Day 2 schedule) was invisible to queries about Stevens speakers.
7. **Why hybrid BM25 + vector retrieval (added June 2026)?** Pure vector search has two systematic failure modes. First, the embedding model has no representation for novel terms (acronyms, technical strings like “InstructEx”, proper names). These terms score identically across all chunks by cosine similarity, so the relevant chunk is never retrieved. Second, when many chunks have near-identical cosine scores, the wrong chunk surfaces. BM25 lexical search solves the first problem via exact term matching; the proper noun bonus ($3\times$ weight) in the reranker solves the second. Reciprocal Rank Fusion merges both lists with a normalized score used as the reranker base, so BM25-only chunks are not penalized by a placeholder similarity value. Tested against 10 hard questions; all answered correctly after implementation.
8. **Why BM25Okapi and not BM25Plus or BM25L?** BM25Okapi is the standard formulation used in most IR literature and is the default in `rank_bm25`. For the KB size (165 chunks), the length normalization differences between variants are negligible. BM25Okapi was sufficient once the RRF score normalization fixed the base-score collapse for BM25-only results. Migrate to BM25Plus if the KB grows significantly and long chunks are consistently under-ranked.

19 Glossary

RAG (Retrieval-Augmented Generation)

Architecture where a retrieval step fetches relevant documents before generation. The LLM is grounded in retrieved facts rather than relying solely on training data.

Embedding

A high-dimensional float vector that represents the semantic meaning of a piece of text. Similar texts produce similar vectors.

Cosine similarity

Distance metric between two vectors measured as the cosine of the angle between them. Value range: $[0, 1]$ after normalization.

Chunk

A fragment of a knowledge base document, sized to fit within a token limit, that becomes one entry in the vector store.

HNSW

Hierarchical Navigable Small World — the approximate nearest neighbour algorithm used by ChromaDB for fast vector search.

Cog discord.py term for a module that groups related commands and event listeners. Loaded and unloaded as a unit.

Ephemeral message

A Discord message visible only to the user who triggered the command, not to the rest of the channel.

Intent detection

Classifying the purpose of an incoming message (greeting, question, food query, etc.) to route it to the right handler without involving the LLM for simple cases.

Reranking

A second scoring pass after initial retrieval. Combines cosine similarity with keyword density, source type, and section title matches to improve the ordering of retrieved chunks.

qa_base_id

Metadata tag added to continuation chunks of a multi-part Q&A answer. Enables the retriever to fetch all sibling parts when any one part is retrieved.

Asymmetric retrieval

Using different prefix strings when embedding queries vs. documents. Exploits model training to better align short questions with longer document passages.

BM25 (Best Match 25)

Probabilistic lexical retrieval algorithm that scores chunks by term frequency and inverse document frequency. Handles exact/sparse term matching that semantic embeddings cannot. Used alongside vector search in hybrid retrieval.

Reciprocal Rank Fusion (RRF)

A method for merging multiple ranked lists. Each item scores $\sum 1/(k + \text{rank})$. Items appearing in both vector and BM25 results receive combined scores; BM25-only items (exact term matches) are still included. $k=60$ is the standard constant.

Hybrid retrieval

Running vector search and BM25 in parallel and fusing the results. Fixes two failure classes: novel/unknown terms and near-equal cosine similarity ties.

Contact: md724@njit.edu · Discord: discord.gg/a4mnbEmSAq · June 2026